

Bloque 16 — Excel, Power Query y entrega automatizada

Propósito

Una hoja de cálculo suele ser el último kilómetro de un análisis: la abre Operaciones, la revisa Finanzas y la usa una persona que no ejecutará tu código. Por eso Excel no es una alternativa a SQL o Python: es una interfaz de entrega con riesgos propios. En este bloque Leo actúa como analista de **Norte Operaciones**, una plataforma de suscripciones. Cada lunes debe entregar un libro con las operaciones pagadas de la semana anterior, excepciones, conciliación y trazabilidad.

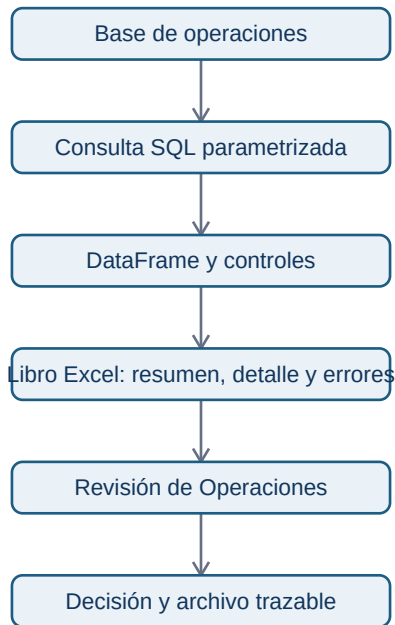
La pregunta continua es: **¿cómo entregar una cifra semanal que otra persona pueda revisar sin repetir pasos manuales ni perder el significado del dato?**

Resultados observables

Al terminar podrás distinguir la herramienta adecuada para cada parte del trabajo, preparar datos repetibles con Power Query, consultar una base de solo lectura desde Python, validar una extracción, generar un libro Excel con varias hojas y dejar registro de parámetros, fuente y errores.

Prerrequisitos: bloques 01, 02, 05, 09 y 13. Se explican desde cero los conceptos propios de Excel, Power Query y una exportación automatizada.

Mapa del caso



El flujo separa **fuentes**, **cálculo**, **control** y **entrega**. Excel puede mostrar un resultado muy convincente aunque la consulta haya usado un periodo incorrecto; por ello los controles y metadatos viajan dentro del mismo libro.

Lecciones

1. De la exportación manual al proceso reproducible
2. Excel profesional: tablas, fórmulas y controles
3. Power Query: importar y transformar sin repetir clics
4. Consultar y validar datos desde Python
5. Generar un libro Excel verificable
6. Automatizar, operar y entregar el informe

Práctica

Resuelve el [informe semanal de operaciones](#) antes de ver la [solución razonada](#). El [script de laboratorio](#) genera un libro real a partir de una base SQLite local; puedes ejecutarlo en Colab u ordenador tras instalar [pandas](#) y [openpyxl](#)¹. El planteamiento también se puede razonar desde el móvil.

Criterio de dominio

No basta con que el archivo abra. Debes poder responder: ¿qué periodo se extrajo?, ¿qué filas se excluyeron?, ¿cuadra el total con la fuente?, ¿quién puede modificar el libro y cómo se volvería a generar el lunes siguiente?

1. De la exportación manual al proceso reproducible

Objetivo y punto de partida

Aprenderás a decidir cuándo basta exportar con un botón y cuándo construir un proceso repetible. Necesitas recordar que una **fila** representa una observación y que el **grano** declara qué representa exactamente. Aquí, una fila representa un intento de cobro, no necesariamente una suscripción ni un cliente.

El problema antes del nombre técnico

El lunes Marta abre una herramienta, filtra fechas, exporta un CSV, borra columnas, pega una tabla en Excel y manda el archivo. La semana siguiente repite los clics. Si cambia una fecha, un filtro o un cálculo, el resultado puede ser plausible pero no repetible. El problema no es que use Excel: es que el procedimiento está solo en su memoria.

Un **proceso reproducible** recibe unas entradas declaradas, ejecuta pasos conocidos y deja las mismas salidas y controles para las mismas entradas. No elimina la revisión humana; permite que la revisión se centre en decisiones y anomalías, no en reconstruir clics.

¿Qué cambia entre una exportación puntual y una entrega que se repite?



La ruta recurrente añade parámetros y evidencia. No es automáticamente mejor: para una pregunta única de veinte filas, el botón puede ser más rápido y suficientemente seguro. Automatiza cuando hay repetición, varios pasos, riesgo de error, necesidad de auditoría o varias consultas que deben ser coherentes.

Elegir la herramienta por responsabilidad

SQL pregunta a una base de datos y reduce el volumen cerca de la fuente. **Python/Pandas** aplica lógica repetible, validaciones y transformaciones que conviene versionar. **Power Query** permite importar y transformar datos de forma visible y refrescable dentro de Excel o Power BI. **Excel** permite revisar, explorar, anotar y entregar un resultado a consumidores de negocio.

No hay una jerarquía universal. Una tabla dinámica es excelente para explorar cientos o miles de filas ya preparadas; no es la fuente de verdad para un cálculo complejo que se debe regenerar cada semana. Tampoco se deben exportar millones de filas a Excel: se agrega o filtra antes, se entrega una muestra/detalle justificado y se conserva la fuente en la base o en un formato analítico.

Error habitual

«La interfaz ya exporta a Excel, así que Python sobra» confunde exportar con controlar. Python no hace mágica a la base: permite parametrizar fechas, ejecutar varias consultas, registrar exclusiones, dar formato coherente y detectar un total inesperado antes de que llegue a dirección.

Resumen y comprobación

Un proceso recurrente declara entradas, transforma de manera conocida, valida y registra la entrega. Antes de automatizar, pregunta: ¿qué decisión se toma?, ¿con qué frecuencia?, ¿qué riesgo tiene una cifra errónea y quién debe poder reconstruirla?

Comprobación: describe una exportación que hoy haces con clics. Identifica un parámetro, un control y una evidencia que añadirías.

2. Excel profesional: tablas, fórmulas y controles

Objetivo y prerequisites

Convertirás un rango de celdas en una entrega revisable. Un **libro** es un archivo con hojas; una **hoja** es una cuadrícula; una **celda** guarda un valor, fórmula o formato. Excel sirve para que una persona examine y use una entrega, no para esconder la lógica esencial de una métrica.

De una lista pegada a una tabla controlable

Si pegas operaciones en celdas sueltas, los filtros y fórmulas pueden no incluir las filas nuevas. Una **tabla estructurada** da nombre al conjunto, conserva encabezados y permite que fórmulas, filtros y tablas dinámicas trabajen sobre columnas con significado. Por ejemplo, `importe_eur` comunica más que «columna F».

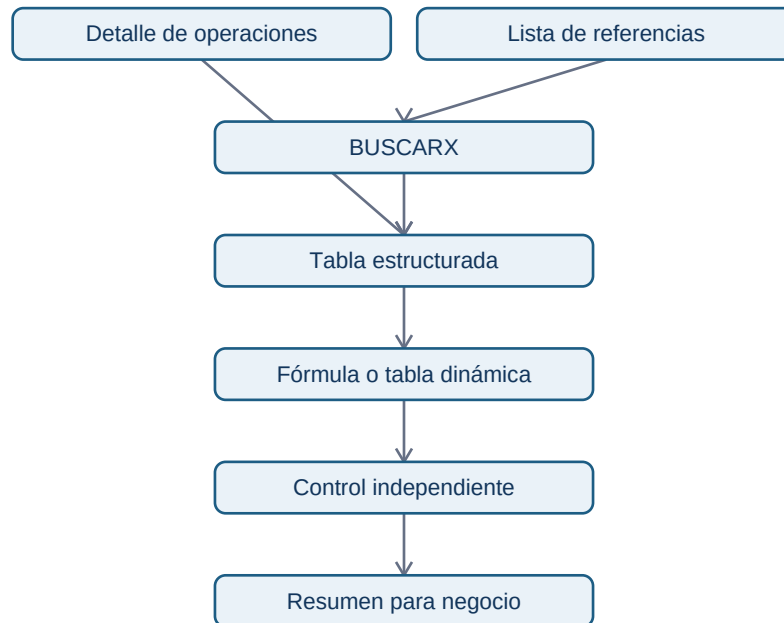
Para el informe de Norte Operaciones, crea una tabla operaciones con `operacion_id`, `fecha_utc`, `estado`, `importe_eur` y `canal`. Filtra por `estado = pagada` para revisar detalle; usa una tabla dinámica para responder «¿cuánto se cobró por canal?»; añade una segmentación si una persona necesita seleccionar canal sin modificar la fuente.

Fórmulas que resuelven preguntas concretas

`SUMAR.SI.CONJUNTO` responde «suma importes que cumplen varias condiciones». Por ejemplo, el total pagado de web en un periodo. `CONTAR.SI.CONJUNTO` permite contar operaciones bajo condiciones. `BUSCARX` busca un valor —por ejemplo, el responsable de un canal— en una tabla de referencia; `INDICE + COINCIDIR` es una alternativa útil cuando se trabaja con versiones antiguas.

Una fórmula no sustituye una definición. Antes de calcular «tasa de rechazo», escribe numerador, denominador, periodo y grano. Si el denominador cuenta intentos y el numerador cuenta operaciones únicas, el porcentaje puede parecer normal y ser inválido.

¿Cómo se protege una entrega contra una cifra incompleta?



El control independiente no debe repetir el mismo error. Si el resumen suma pagos, compara también número de filas, importe contra la extracción y periodo mínimo/máximo. Resalta con formato condicional una diferencia no nula; la celda roja no demuestra que haya un problema, obliga a investigarlo.

Fechas, validación y protección

Una fecha de Excel es un valor con formato, no texto decorativo. Declara zona horaria y límite de periodo: «semana anterior cerrada en UTC» evita que el lunes incluya una hora incompleta. Usa validación de datos para campos manuales como `revisado_por` o `estado_revision`; no permitas que cada persona escriba variantes como “OK”, “okey” o “correcto”.

Congela encabezados, aplica formato de moneda y fecha coherente, protege las celdas de fórmulas y deja editables solo las celdas de comentario si procede. La protección de hoja evita errores accidentales, no es un sistema de seguridad ni sustituye permisos de acceso al archivo.

Límite y comprobación

Excel no es una base transaccional ni un lugar apropiado para millones de filas, secretos o transformaciones críticas sin historial. Úsalo para revisión y entrega; conserva la lógica repetible en consulta, Power Query o código.

Comprobación: ¿qué diferencia hay entre una tabla dinámica y la fuente que alimenta la tabla dinámica? ¿Qué control añadirías a un total de cobros?

Fuente primaria: [Microsoft Learn: Power Query](#) explica el motor de importación y preparación que se utilizará en la siguiente lección.

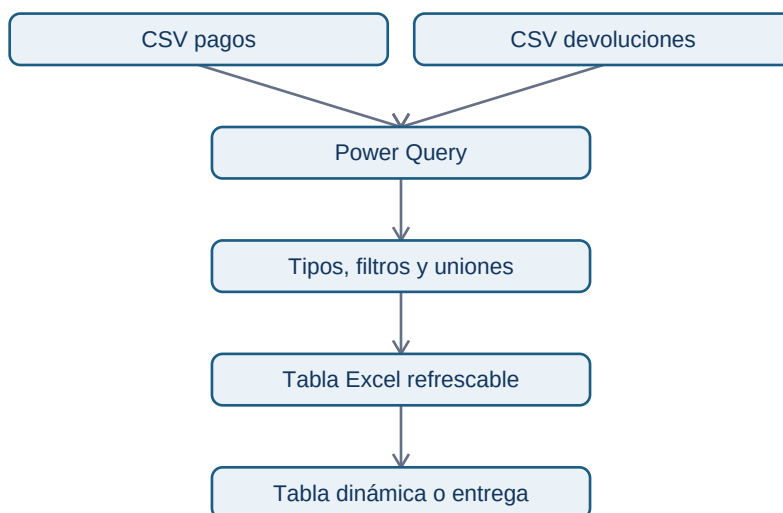
3. Power Query: importar y transformar sin repetir clics

Objetivo

Aprenderás qué aporta Power Query y cómo diseñar una transformación que se pueda refrescar. Power Query es un motor de conexión y preparación de datos disponible, entre otros productos, en Excel y Power BI. Su interfaz registra una secuencia de pasos; no es simplemente «limpiar una vez y guardar».

Caso visible

Norte Operaciones recibe cada semana un CSV de pagos y otro de devoluciones. Ambos tienen nombres de columnas distintos, importes con coma decimal y una fila de prueba que no debe entrar en el informe. Abrir, corregir y pegar cada archivo puede funcionar una semana; al refrescar una consulta, Power Query vuelve a aplicar los pasos documentados al archivo nuevo.



El diagrama representa una preparación para consumo, no una licencia para mezclar cualquier fuente. Antes de anexar archivos comprueba que comparten grano, significan lo mismo y que las columnas ausentes son tratadas explícitamente.

Pasos guiados

1. Usa **Datos** → **Obtener datos** y selecciona el CSV. Guarda el archivo original sin editar: es evidencia de entrada.
2. En el editor, nombra la consulta por su propósito, por ejemplo pagos_semana.
3. Promueve encabezados solo después de verificar que la primera fila contiene nombres. Cambia tipos: fecha a fecha/hora, importe a decimal, identificador a texto para no perder ceros iniciales.
4. Filtra registros de prueba con una regla visible; no borres filas «porque parecen raras» sin criterio.
5. Combina consultas con **Anexar** cuando son el mismo tipo de hecho y con **Combinar** cuando agregas atributos mediante una clave. Revisa la cardinalidad de la clave antes de expandir columnas.
6. Carga como tabla y refresca con otro archivo de muestra. Si el refresco falla, el fallo es información: puede haber cambiado el esquema o el formato.

Cuándo usar Power Query, Python o SQL

Power Query es apropiado cuando el consumidor necesita ver o mantener una preparación sencilla en Excel/Power BI y el volumen cabe en ese flujo. SQL es preferible para filtrar, agregar y unir datos en una base con permisos y volumen. Python es preferible para reglas complejas, pruebas automáticas, llamadas a API, generación de archivos y versionado. Es habitual combinarlos, pero hay que asignar una fuente de verdad a cada regla.

Error frecuente: el refresco que cambia el resultado

Si una columna pasa de `importe` a `importe_total`, una consulta puede fallar o, peor, cargar nulos. Añade controles de número de filas, columnas esperadas, fechas mínima/máxima y total. Power Query permite pasos como quitar errores o reemplazar nulos; no los apliques sin medir cuántos registros afectan y sin registrar la decisión.

Resumen y práctica

Power Query hace explícitos los pasos de importación y transformación, por eso permite refrescar. No reemplaza la comprensión del grano, los tipos ni la validación.

Práctica: diseña la secuencia de consultas para pagos y devoluciones. Escribe qué clave verificarías antes de combinarlas y qué control ejecutarías después.

Fuente primaria: [Microsoft Learn: qué es Power Query](#) documenta sus conectores, transformaciones y límites de producto.

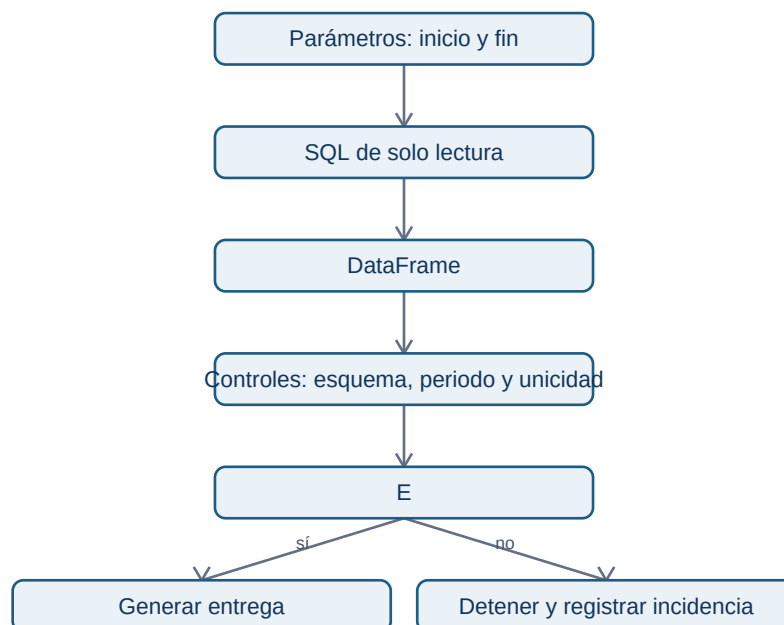
4. Consultar y validar datos desde Python

Objetivo

Extraerás datos de una base con una consulta de solo lectura, parámetros y controles. Una **consulta parametrizada** separa la instrucción SQL de los valores como fechas; evita construir texto SQL mezclando datos externos y hace visible qué periodo se pidió.

Antes de programar: contrato de extracción

Para el informe semanal define: variable objetivo = importes cobrados; grano = un intento de cobro; periodo = desde el lunes 00:00 UTC inclusive hasta el lunes siguiente exclusivo; fuente = tabla operaciones; exclusión = pruebas internas; salida = resumen, detalle y rechazados. Sin este contrato, el código puede ser correcto y responder otra pregunta.



Detener es una salida válida. Generar un Excel con una extracción incompleta solo porque el script no produjo una excepción es peor que avisar de una incidencia.

Ejemplo mínimo

```
import sqlite3
import pandas as pd

consulta = """
SELECT operacion_id, fecha_utc, estado, importe_eur, canal
FROM operaciones
```

```
WHERE fecha_utc >= :inicio AND fecha_utc < :fin
AND es_prueba = 0
"""

with sqlite3.connect("operaciones.sqlite") as conexion:
    datos = pd.read_sql_query(
        consulta, conexion,
        params={"inicio": "2026-07-13", "fin": "2026-07-20"},
    )
```

`read_sql_query` crea un `DataFrame` —una tabla en memoria con filas y columnas— a partir de la consulta. SQLite es útil para aprender porque es local; en un entorno empresarial el conector puede ser PostgreSQL u otro motor. Las credenciales no se escriben dentro del script ni se suben al repositorio: se inyectan mediante variables de entorno o un gestor de secretos y se usan con un usuario de solo lectura.

Controles que responden a riesgos

- **Esquema:** ¿están las columnas necesarias y sus tipos son interpretables?
- **Periodo:** ¿la fecha mínima y máxima están dentro de los límites declarados?
- **Unicidad:** ¿operacion_id se repite cuando el grano promete una fila por intento?
- **Compleitud:** ¿hay nulos en identificador, fecha, estado o importe?
- **Conciliación:** ¿el número e importe de pagos concuerda con una consulta independiente o total de control?
- **Exclusiones:** ¿cuántas filas de prueba, devoluciones o estados desconocidos quedaron fuera y por qué?

Un `assert` o una excepción con mensaje claro puede impedir la entrega. Un control no es un adorno: debe estar vinculado a una decisión de qué hacer al fallar.

Límite técnico y ético

Parametrizar valores no habilita parametrizar arbitrariamente nombres de tabla o permisos. No ejecute escrituras (`DELETE`, `UPDATE`) desde un informe; limita columnas y filas al mínimo necesario y evita exportar identificadores personales si el destinatario no los requiere.

Comprobación: si el total cae a cero porque cambió el nombre de un estado, ¿qué control lo detectaría y qué debería hacer el proceso?

Fuente primaria: `pandas read_sql_query`.

5. Generar un libro Excel verificable

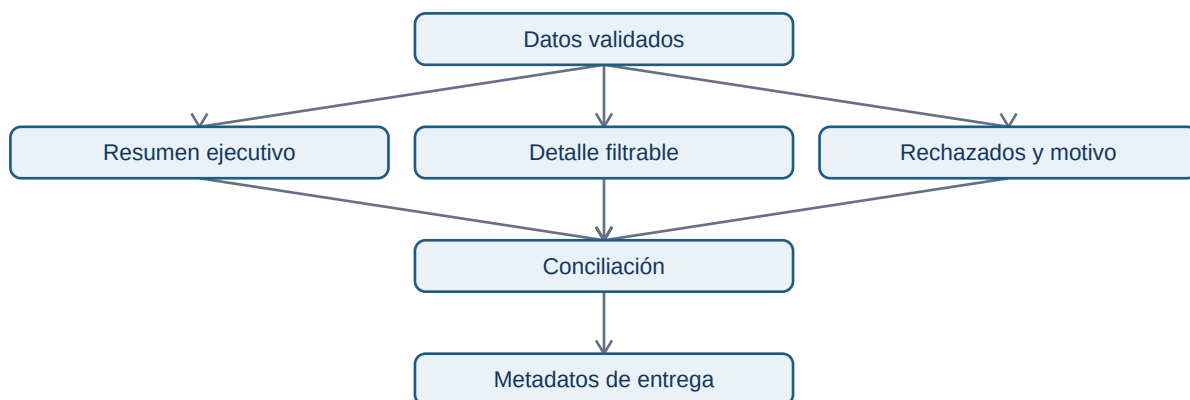
Objetivo

Construirás un archivo que sirva para leer y revisar, no solo para descargar filas.

`DataFrame.to_excel` escribe una tabla; una biblioteca como `openpyxl` permite aplicar formato, congelar encabezados, crear varias hojas y proteger contra modificaciones accidentales.

Diseño antes del código

El libro semanal de Norte Operaciones tendrá cinco hojas: Resumen, Detalle, Rechazados, Conciliación y Metadatos. Esta separación evita que el resumen esconda excepciones y permite a cada audiencia empezar por la hoja adecuada.



La hoja de metadatos registra fecha de generación, parámetros, fuente, versión del script y controles ejecutados. No es una garantía de veracidad: es el punto de partida para que alguien reconstruya una cifra.

Ejemplo de generación

```
from pathlib import Path
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill

ruta = Path("salidas/informe_operaciones_2026-07-13.xlsx")
with pd.ExcelWriter(ruta, engine="openpyxl") as escritor:
    resumen.to_excel(escritor, sheet_name="Resumen", index=False)
    detalle.to_excel(escritor, sheet_name="Detalle", index=False)
    rechazados.to_excel(escritor, sheet_name="Rechazados", index=False)
    controles.to_excel(escritor, sheet_name="Conciliacion", index=False)
    metadatos.to_excel(escritor, sheet_name="Metadatos", index=False)

libro = load_workbook(ruta)
for hoja in libro.worksheets:
    hoja.freeze_panes = "A2"
```

```
hoja.auto_filter.ref = hoja.dimensions
for celda in hoja[1]:
    celda.font = Font(bold=True, color="FFFFFF")
    celda.fill = PatternFill("solid", fgColor="1D5D84")
libro.save(ruta)
```

El formato sigue al significado: `importe_eur` usa moneda; `fecha_utc` usa un formato de fecha/hora y un nombre que declare la zona; las columnas se ajustan con límite razonable para que no aparezcan hojas ilegibles. Evita fórmulas críticas ocultas; si introduces una, documenta fórmula, rango y control independiente.

Conciliación y lectura humana

En Conciliación, compara al menos: filas extraídas, identificadores distintos, importe de estados pagados, importe de devoluciones y diferencia contra el total de la consulta de control. Muestra resultado, umbral y acción: «diferencia = 0 → entregar; diferencia ≠ 0 → bloquear y revisar». Los registros rechazados deben conservar motivo y regla de rechazo, no desaparecer.

Errores frecuentes

Un archivo con formato bonito puede ser inutilizable si: cambia la columna de fecha a texto, trunca decimales, exporta filas personales no necesarias, sobrescribe el informe anterior o titula “Total” a una suma que mezcla monedas. El nombre de archivo debe incluir periodo y momento de generación, por ejemplo `operaciones_2026-07-13_a_2026-07-20_generado_2026-07-20T0815Z.xlsx`.

Resumen

Un libro profesional comunica resultado, detalle, excepciones y evidencia. Formatear no es maquillar: reduce errores de interpretación y facilita una revisión que pueda fallar de manera visible.

Fuente primaria: [documentación de `DataFrame.to\_excel`](#) y [tutorial de `openpyxl`](#).

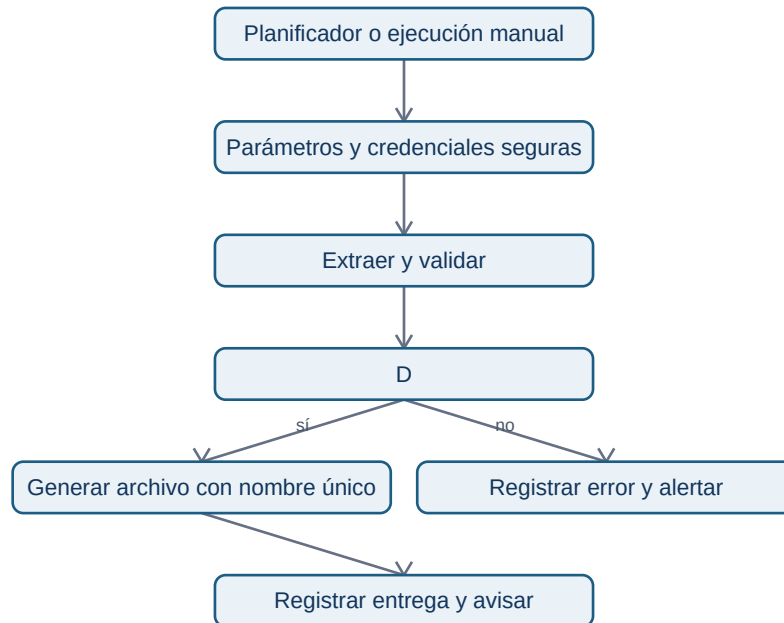
6. Automatizar, operar y entregar el informe

Objetivo

Convertirás el script en una operación fiable: entradas explícitas, registro, errores visibles y una salida que no destruya evidencias anteriores. Automatizar no significa ejecutar sin supervisión; significa poder saber qué ocurrió cuando algo sale distinto.

El contrato operativo

Cada ejecución recibe --inicio, --fin y --salida o valores equivalentes. Registra versión del script, hora UTC, consulta usada, número de filas, resultados de controles y ruta del archivo. Devuelve código de salida distinto de cero si no puede conectarse, faltan columnas o falla una conciliación. Un planificador —por ejemplo, una tarea del sistema o una automatización corporativa— debe alertar ante ese fallo, no enviar un archivo vacío.



El planificador no sustituye el juicio analítico. Si un cambio de negocio hace que los rechazos suban, el informe puede ser correcto y aun así requerir una explicación antes de distribuirlo.

Reglas de operación

- Conserva archivos de entrada y salidas según la política de retención; no sobrescribas el lunes anterior.
- Usa rutas configurables y un directorio de salida separado del código.
- Nunca guardes contraseñas, tokens ni datos personales de prueba en Git. Un archivo .env local puede aportar configuración, pero también se excluye del repositorio.
- Registra valores seguros: periodo, conteos, duración y mensaje de error. No registres datos personales o secretos.
- Prueba primero una semana histórica conocida y compara con una conciliación manual independiente.
- Documenta el propietario, la cadencia, el destinatario y qué hacer cuando falla un control.

Ejemplo de informe útil

El director de Operaciones abre Resumen: importe cobrado, variación frente a semana comparable y estado de controles. Si hay diferencia, no usa ese total para decidir. El equipo analista abre Conciliación y Rechazados, identifica si el origen es un estado nuevo o una carga tardía, corrige o explica la excepción y deja registro. Esta es la diferencia entre enviar un Excel y entregar un proceso.

Autoevaluación y siguiente paso

Comprueba: ¿podría otra persona generar el mismo informe con la misma base?, ¿sabría si el periodo fue mal introducido?, ¿podría distinguir un fallo técnico de una variación real?, ¿el destinatario ve solo lo que necesita?

Resuelve ahora el [proyecto de informe semanal](#). En el siguiente ciclo de profesionalización, este mismo contrato se conectará con un modelo dimensional y un dashboard BI, sin duplicar definiciones.